

Conduction of Semester End Practical Examination:

1. All laboratory experiments are to be included for practical examination.
2. Students are allowed to pick one experiment from Part A and Part B with lot.
3. Strictly follow the instructions as printed on the cover page of answer script.
4. Marks distribution : Procedure + Conduction + Viva : 50
 Part A : 5 + 15 + 5 = 25
 Part B : 5 + 15 + 5 = 25
5. Change of experiments is allowed only once and marks allotted to the procedure part to be made zero.

Part A

1. Program to implement frame-sorting.

- Each frame contains sequence number, time and message.
- Sequence number must be a random number of 2 digits, Regenerate the repeated Sequence number.
- Sort the frames based on sequence number using bubble sort technique.
- Display all the information of the packet before sort and after sorting neatly.

```
#include<stdio.h>
```

```
struct packet
{
int seqno;
char a[100];
float time;
}
p[100],temp;
```

```
void bubblesort(int n)
{
int i,j;
for(i=0; i<n-1; i++)
{
for(j=0; j<n-i-1; j++)
{
if(p[j].seqno>p[j+1].seqno)
{
temp=p[j+1];
p[j+1]=p[j];
p[j]=temp;
}
}
}
}
```

```
void main()
{
int i,j,n,num;
printf("Enter the number of frames:");
scanf("%d",&n);
for(i=0; i<n; i++)
{
label:
num=rand()%100;
for(j=0; j<=i; j++)
{
```

```

    if(p[j].seqno==num)
    {
        goto slabel;
    }
}
p[i].seqno=num;
printf("Enter message:");
scanf("%s",&p[i].a);
p[i].time=rand();
}

printf("Before sorting:\n");
for(i=0; i<n; i++)
{
    printf("%d\t%s\t%f\n",p[i].seqno,p[i].a,p[i].time);
}
bubblesort(n);
printf("After sorting:\n");
for(i=0; i<n; i++)
{
    printf("%d\t%s\t%f\n",p[i].seqno,p[i].a,p[i].time);
}
}

```

2. Program to implement frame-sorting. Each frame contains sequence number, time and message. Sequence number must be a random number of 4 digits. Sort the frames based on sequence number using quick sort technique.

```

#include<stdio.h>
#include<stdlib.h>

struct packet
{
    int seqno;
    char a[100];
    float time;
}
p[100],temp;

void quicksort(int left,int right)
{
    int i,j,pivot;
    if(left<right)
    {
        i=left;
        j=right+1;
        pivot=p[left].seqno;

```

```

do
{
do
{
i++;
}

while(p[i].seqno<pivot&& i<=right);
do
{
j--;
}

while(p[j].seqno>pivot&& j>=left);
if(i<j)
{
temp=p[i];
p[i]=p[j];
p[j]=temp;
}
}

while(i<=j);
temp=p[left];
p[left]=p[j];
p[j]=temp;
quicksort(left,j-1);
quicksort(j+1,right);
}
}

void main()
{
int i,j,n,num;
printf("Enter the number of frames:");
scanf("%d",&n);
for(i=0; i<n; i++)
{
slabel:
num=rand()%10000;
for(j=0; j<i; j++)
{
if(p[j].seqno==num)
goto slabel;
}
p[i].seqno=num;
printf("Enter message:");
scanf("%s",p[i].a);

```

```

p[i].time=rand();
}

printf("Before sorting:\n");
for(i=0; i<n; i++)
{
printf("%d\t%s\t%f\n",p[i].seqno,p[i].a,p[i].time);
}

quicksort(0,n-1);
printf("After sorting:\n");
for(i=0; i<n; i++)
{
printf("%d\t%s\t%f\n",p[i].seqno,p[i].a,p[i].time);
}
}

```

3. Program to implement frame-sorting.

- Each frame contains sequence number, packet ID, Source IP Address, Destination IP Address, port number and message.
- Sequence number must be a random number of 3 digits, Regenerate the repeated Sequence number.
- Sort the frames based on sequence number using insertion sort technique.
- Display all the information of the packet neatly.

```

#include<stdio.h>

struct packet
{
int seqno;
int ponum;
char sip[20];
char dip[20];
int pid;
char a[100];
float time;
}
p[100],temp;

void insertionsort(int n)
{
int i,j;
for(i=1; i<n-1; i++)
{
temp=p[i];
for(j=i-1; j>=0&& temp.seqno<p[j].seqno; j--)
{

```

```

p[j+1]=p[j];
}
p[j+1]=temp;
}
}

void main()
{
int i,j,n,num;
printf("Enter the number of frames:");
scanf("%d",&n);
for(i=0; i<n; i++)
{
label:
num=rand()%1000;
for(j=0; j<=i; j++)
{
if(p[j].seqno==num)
{
goto label;
}
}
p[i].seqno=num;
printf("Enter message:");
scanf("%s",&p[i].a);
printf("Enter packet id:");
scanf("%d",&p[i].pid);
printf("Enter port number:");
scanf("%d",&p[i].ponum);
printf("Enter source ip:");
scanf("%s",&p[i].sip);
printf("Enter destination ip:");
scanf("%s",&p[i].dip);
}

printf("Before sorting:\n");
for(i=0; i<n; i++)
{
printf("%d\t%d\t%d\t%s\t%s\t%s\n",p[i].seqno,p[i].pid,p[i].ponum,p[i].sip,p[i].dip,p[i].a);
}

insertionsort(n);
printf("After sorting:\n");
for(i=0; i<n; i++)
{
printf("%d\t%d\t%d\t%s\t%s\t%s\n",p[i].seqno,p[i].pid,p[i].ponum,p[i].sip,p[i].dip,p[i].a);
}

```

```
}  
}
```

4. Write a program for error detection code using Parity check technique.

*****SENDER*****

Enter Message : 1011

1. ODD parity

2. even parity

Enter a option : 1

Message to be Transmitted : 10110

Do You Want to Introduce error(Y/N) : Y

Enter the Position : 1 (condition : position between 1 to T size)

*****RECEIVER*****

Message received at the Receiver : 00110

ERROR in MESSAGE

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char message[100], transmitted[100];
```

```
    int choice, i, len, parityBit = 0, count = 0;
```

```
    char errorOption;
```

```
    int errorPos;
```

```
    // ***** SENDER *****
```

```
    printf("***** SENDER *****\n");
```

```
    printf("Enter Message : ");
```

```
    scanf("%s", message);
```

```
    len = strlen(message);
```

```
    // Count number of 1s
```

```
    for (i = 0; i < len; i++) {
```

```
        if (message[i] == '1')
```

```
            count++;
```

```
    }
```

```
    printf("1. ODD parity (0)\n2. EVEN parity (1)\n");
```

```
    printf("Enter a option : ");
```

```
    scanf("%d", &choice);
```

```
    // Decide parity bit
```

```
    if (choice == 1) { // Odd parity
```

```

    parityBit = (count % 2 == 0) ? 1 : 0;
} else {          // Even parity
    parityBit = (count % 2 == 0) ? 0 : 1;
}

// Form transmitted message
strcpy(transmitted, message);
transmitted[len] = parityBit + '0';
transmitted[len + 1] = '\0';

printf("Message to be Transmitted : %s\n", transmitted);

// Error introduction
printf("Do You Want to Introduce error(Y/N) : ");
scanf(" %c", &errorOption);

if (errorOption == 'Y' || errorOption == 'y') {
    printf("Enter the Position : ");
    scanf("%d", &errorPos);

    if (errorPos >= 1 && errorPos <= strlen(transmitted)) {
        // Flip the bit at errorPos
        transmitted[errorPos - 1] =
            (transmitted[errorPos - 1] == '0') ? '1' : '0';
    }
}

// ***** RECEIVER *****
printf("\n***** RECEIVER *****\n");
printf("Message received at the Receiver : %s\n", transmitted);

// Check parity
count = 0;
for (i = 0; i < strlen(transmitted); i++) {
    if (transmitted[i] == '1')
        count++;
}

if (choice == 1) { // Odd parity
    if (count % 2 == 1)
        printf("NO ERROR in MESSAGE\n");
    else
        printf("ERROR in MESSAGE\n");
} else { // Even parity
    if (count % 2 == 0)
        printf("NO ERROR in MESSAGE\n");
    else
        printf("ERROR in MESSAGE\n");
}

```



```

}

return 0;
}

```

5. Write a program for error detecting code using CRC technique

*****SENDER*****

Enter Message : 101010

Enter Pattern : 1111 (Condition : pattern size should be less than Message or else re enter pattern)

Given Message is : 101010

Given Pattern is : 1111

Message size is : 6

Pattern Size is : 4

FCS : 3bit

After Appending Q is :101010000

Remainder R is :100

Message to be Transmitted T is :101010**100**

Do You Want to Introduce error(Y/N) : Y

Enter the Position : 2 (condition : position between 1 to T size)

*****RECEIVER*****

Message received at the Receiver : 1**1**1010100

Remainder R is : 100

ERROR IN MESSAGE

```
#include<stdio.h>
```

```
#include<string.h>
```

```
void crc(char message[], char pattern[])
```

```
{
```

```
int msg_length = strlen(message);
```

```
int pattern_length = strlen(pattern);
```

```
int fcs_length = pattern_length - 1;
```

```
char msg[20];
```

```
strcpy(msg,message);
```

```
//Performing CRC division
```

```
int i;
```

```
for ( i = 0; i < msg_length; i++)
```

```

{
if (message[i] == '1' )
{
if(i<=msg_length-pattern_length)
{
for (int j = 0; j < pattern_length; j++)
{
if (message[i+j] == pattern[j])
message[i+j] = '0';
else
message[i+j] = '1';
}
}
else break;
}
}

printf("Remainder R is : %s\n",message);
for (; i < msg_length + fcs_length; i++)
{
msg[i]=message[i];
}

printf("Message to be Transmitted T is : %s\n",msg);
char choice;
int position;
printf("\nDo You Want to Introduce error(Y/N): ");
scanf(" %c", &choice);

if (choice == 'Y' || choice == 'y')
{
x:
printf("Enter the Position: ");
scanf("%d", &position);

if (position < 1 || position > strlen(msg))
{
printf("Invalid position. Please enter a position between 1 and %d.\n", strlen(msg));
goto x;
}

//Introducing error at the specified position
msg[msg_length - position] = (msg[msg_length - position] == '1') ? '0' : '1';
}

printf("*** RECEIVER ***");
printf("\nMessage received at the Receiver :%s\n", msg);
for ( i = 0; i < msg_length; i++)

```

```

{
if (msg[i] == '1' )
{
if(i<=msg_length-pattern_length)
{
for (int j = 0; j < pattern_length; j++)
{
if (msg[i+j] == pattern[j])
msg[i+j] = '0';
else
msg[i+j] = '1';
}
}
else break;
}
}

printf("Remainder R is : %s\n",msg);
int flag=0;

for(i=0;i<msg_length;i++)
{
if(msg[i]=='1')
{
flag=1;
break;
}
}

if (flag)
{
printf("ERROR IN MESSAGE");
}
else
printf("MESSAGE WITHOUT ERROR");
}

void main()
{
char message[100];
char pattern[100];
char choice;
int position;

printf("*** SENDER ***");
printf("\nEnter Message: ");
scanf("%s", message);

```

```

printf("Enter Pattern: ");
scanf("%s", pattern);

int msg_length = strlen(message);
int pattern_length = strlen(pattern);

if (pattern_length >= msg_length)
{
printf("Pattern size should be less than Message. Please re-enter the Pattern \n");
return 0;
}

printf("Given Message is : %s\n", message);
printf("Given Pattern is : %s\n", pattern);
printf("Message size is : %d\n", msg_length);
printf("Pattern Size is : %d\n", pattern_length);
printf("FCS          : %d bit\n", pattern_length - 1);

//Appending zeros to the message
for (int i = 0; i < pattern_length - 1; i++)
{
message[msg_length + i] = '0';
}
message[msg_length + pattern_length - 1] = '\0';

printf("After Appending Q is: %s\n", message);
crc(message, pattern);
}

```

6. Write a program to implement Leaky Bucket algorithm.

Enter the Size of the Bucket : 20

Enter the out rate : 5

Enter the time Interval : 5

Enter the number of Packet : 5

Enter the time and Size of Packet 1 : 2 6

Enter the time and Size of Packet 2 : 3 10

Enter the time and Size of Packet 3 : 14 4

Enter the time and Size of Packet 4 : 16 15

Enter the time and Size of Packet 5 : 21 10

Operation	Time	Filled	Free-Space
Insert	2	6	14
Insert	3	16	4
Remove	5	11	9
Remove	10	6	14
Insert	14	10	10
Remove	15	5	15
Insert	16	20	0
Remove	20	15	5
Insert	21	overflow	
Remove	25	10	10
Remove	30	5	15
Remove	35	0	20

```
#include<stdio.h>
```

```
struct ts
```

```
{
```

```
int t;
```

```
int ps;
```

```
}
```

```
s[20];
```

```
void main()
```

```
{
```

```
int n,outrate,time,np,i;
```

```
printf("Enter the size of the bucket:");
```

```
scanf("%d",&n);
```

```
printf("Enter the outrate of the bucket:");
```

```

scanf("%d",&outrate);

printf("Enter the time interval of leak:");
scanf("%d",&time);

printf("Enter the number of packets:");
scanf("%d",&np);

for(i=0;i<np;i++)
{
printf("Enter the time and size of packet%d: ",i);
scanf("%d",&s[i].t);
scanf("%d",&s[i].ps);
}

printf("oper time filled free-space\n");
int t[30],filled[30],free[30],k=1,j=0,flag;
i=0;

do
{
flag=0;
if(i>=np)
{
l:
{
printf("remove ");
t[j]=time*k;
filled[j]=filled[j-1]-outrate;

printf("%d  %d  ",t[j],filled[j]);
k++;
}
}

//inserting
else if(s[i].t<time*k){
printf("insert ");
t[j]=s[i].t;

if(j==0 && s[i].ps<=n)
{
filled[j]=s[i].ps;
printf("%d  %d  ",t[j],filled[j]);
}

else if((filled[j-1]+s[i].ps)<=n)
{

```

```

filled[j]=filled[j-1]+s[i].ps;
printf("%d    %d    ",t[j],filled[j]);
}

else
{
flag=1;
printf("%d    overflow    ",t[j]);
}
}

else
{
i--;
goto l;
}

if(flag==0)
{
free[j]=n-filled[j];
printf("%d\n",free[j]);
}

else
{
printf("\n");
filled[j]=filled[j-1];
}

j++;
i++;
}
while(filled[j-1]!=0);
}

```

7. Write a program to find the shortest path using Dijkstra's Algorithm

Enter a number of Node : 4

Enter distance to A to A : 0

Enter distance to A to B : 10

Enter distance to A to C : 0

Enter distance to A to D : 0

Enter distance to B to A : 10

Enter distance to B to B : 0

Enter distance to B to C : 3

Enter distance to B to D : 20
Enter distance to C to A : 0
Enter distance to C to B :3
Enter distance to C to C :0
Enter distance to C to D :10
Enter distance to D to A :0
Enter distance to D to B :20
Enter distance to D to C :10
Enter distance to D to D :0

Enter starting Node - A

Distance of Node B - 10
Route A -> B
Distance of Node C - 13
Route A -> B→ C
Distance of Node D - 23
Route A -> B ->C->D

Enter a Destination Node : C

The shortest route is - A -> B →C
Distance is 13

```
#include<stdio.h>
#define INFINITY 9999
#define MAX 100

void Dijkstra(int cost[MAX][MAX], int n, int start, int destination)
{
    int distance[MAX], pred[MAX];
    int visited[MAX], count, mindistance, nextnode, i, j;

    // Creating cost matrix
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (cost[i][j] == 0)
                cost[i][j] = INFINITY;

            else
                cost[i][j] = cost[i][j];
        }
    }
}
```



```

for (i = 0; i < n; i++)
{
    distance[i] = cost[start][i];
    pred[i] = start;
    visited[i] = 0;
}

distance[start] = 0;
visited[start] = 1;
count = 1;

while (count < n - 1)
{
    mindistance = INFINITY;

    for (i = 0; i < n; i++)
    {
        if (distance[i] < mindistance && !visited[i])
        {
            mindistance = distance[i];
            nextnode = i;
        }
    }

    visited[nextnode] = 1;
    for (i = 0; i < n; i++)
    {
        if (!visited[i])
        {
            if (mindistance + cost[nextnode][i] < distance[i])
            {
                distance[i] = mindistance + cost[nextnode][i];
                pred[i] = nextnode;
            }
        }
    }
    count++;
}

// Printing the distance of each node from the source
for (i = 0; i < n; i++)
{
    if (i != start)
    {
        printf("Distance of Node %d: %d\n", i, distance[i]);
        printf("Route: %d", start);
        int path = pred[i];
        while (path != start)

```

```

{
printf(" --> %d ", path);
path = pred[path];
}
printf(" --> %d\n", i);
}
}
// Printing the shortest path
printf("Shortest distance from source %d to destination %d: %d\n", start, destination, distance[destination]);
printf("Shortest path: %d ", start);
int path = pred[destination];
while (path != start)
{
printf(" --> %d ", path);
path = pred[path];
}
printf(" --> %d\n", destination);
}
int main()
{
int cost[MAX][MAX], i, j, n, u, destination;
printf("Enter the number of nodes:");
scanf("%d", &n);
for (i = 0; i < n; i++)
{
for (j = 0; j < n; j++)
{
if (i > j)
{
printf("Distance between %d and %d: %d\n", i, j, cost[j][i]);
cost[i][j] = cost[j][i];
}
else
{
printf("Distance between %d and %d:", i, j);
scanf("%d", &cost[i][j]);
}
}
}
printf("Enter the starting node:");
scanf("%d", &u);

printf("Enter the destination node:");
scanf("%d", &destination);

Dijkstra(cost, n, u, destination);
return 0;

```

}

8. Write a program to construct distance vector table for all the router of a given network using Bellman-Ford Distance Vector Routing protocol method.

Example:

Enter the number of nodes : 4

Enter the cost matrix:

```
 1  2  3  4
1 0  3  5 999
2 3  0 999 1
3 5  4  0  2
4 999 1  2  0
```

Intial Table

Router 1

Destination	Distance	Next Hop
1	0	1
2	3	2
3	5	3
4	999	-

Similarly construct for Router 2, 3 ,4.

Updated table

Router 1

Destination	Distance	Next Hop
1	0	1
2	3	2
3	5	3
4	4	2

Router 2

Destination	Distance	Next Hop
1	3	1
2	0	2
3	3	4
4	1	4

Router 3

Destination	Distance	Next Hop
1	5	1
2	3	4
3	0	3
4	2	4

Router 4

Destination	Distance	Next Hop
1	4	2
2	1	2
3	2	3

4	0	4
---	---	---

```
#include <stdio.h>
```

```
struct node
{
unsigned dist[20];
unsigned from[20];
}
rt[10];
```

```
int main()
{
int dmat[20][20];
int n, i, j, k, count = 0;
```

```
printf("Enter the number of nodes: ");
scanf("%d", &n);
```

```
printf("\nEnter the cost matrix:(Assign value 999 if there is no direct connection)\n");
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
{
scanf("%d", &dmat[i][j]);
dmat[i][i] = 0;
rt[i].dist[j] = dmat[i][j];
rt[i].from[j] = j;
}
```

```
printf("\nInitial Tables:\n");
for (i = 0; i < n; i++)
{
printf("\nRouter %d:\n", i + 1);
printf("Destination\tDistance\tNext Hop\n");
```

```
for (j = 0; j < n; j++)
{
if(rt[i].dist[j]==999)
{
printf("%d\t%d\t\t\n", j + 1, rt[i].dist[j]);
}
else
```

```

{
printf("%d\t\t%d\t\t%d\n", j + 1, rt[i].dist[j], rt[i].from[j] + 1);
}
}
}

do
{
count = 0;
for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
for (k = 0; k < n; k++)

if (rt[i].dist[j] > dmat[i][k] + rt[k].dist[j])
{
rt[i].dist[j] = rt[i].dist[k] + rt[k].dist[j];
rt[i].from[j] = k;
count++;
}
}

while (count != 0);
printf("\nUpdated Tables:\n");

for (i = 0; i < n; i++)
{
printf("\nRouter %d:\n", i + 1);
printf("Destination\tDistance\tNext Hop\n");

for (j = 0; j < n; j++)
{
printf("%d\t\t%d\t\t%d\n", j + 1, rt[i].dist[j], rt[i].from[j] + 1);
}
}

printf("\n");
return 0;
}

```

Part B : Simulation programs

1. Simulate a four node point-to-point network, and connect the links as follows: n0-n2, n1-n2, n2-n3 and n1-n3.

- Apply UDP traffic between n0-n3.
- Apply relevant applications over UDP agents changing the parameter.
- Set the queue size vary the bandwidth and find the number of packets dropped by UDP.

```
set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set tf [open out.tr w]
$ns trace-all $tf
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit(0)
}
```

```
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
$ns duplex-link $n0 $n2 100Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 5ms DropTail
$ns duplex-link $n2 $n3 1Mb 5ms DropTail
$ns duplex-link $n1 $n3 1Mb 5ms DropTail
$ns queue-limit $n0 $n2 50
$ns queue-limit $n1 $n2 50
$ns queue-limit $n2 $n3 50
$ns queue-limit $n1 $n3 50
set udp1 [new Agent/UDP]
$ns attach-agent $n0 $udp1
set null [new Agent/Null]
$ns attach-agent $n3 $null
```

```
$ns connect $udp1 $null
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packet-size 200
$cbr1 set interval 0.005
$cbr1 attach-agent $udp1
```

```
$ns at 0.5 "$cbr1 start"
```

```
$ns at 5.0 "$cbr1 stop"  
$ns at 5.5 "finish"  
$ns run
```

2. Simulate a six node point-to-point network, and connect the links as follows: n1-n5, n2-n3, n2-n4, n1-n6 and n3-n6.

- **Apply FTP agent between n3-n5 and TELNET between n4-n6.**
- **Apply relevant applications over TCP agents changing the parameter.**
- **Set the queue size vary the bandwidth and find the number of packets dropped by TCP.**
- **Change node color, Link Color, Packet Color, change node position and shape.**

```
set ns [new Simulator]  
$ns color 1 violet  
set nf [open out.nam w]  
$ns namtrace-all $nf  
set tf [open out.tr w]  
$ns trace-all $tf
```

```
proc finish {} {  
    global ns nf tf  
    $ns flush-trace  
    close $nf  
    close $tf  
    exec nam out.nam &  
    exit(0)  
}
```

```
set n1 [$ns node]  
set n2 [$ns node]  
set n3 [$ns node]  
set n4 [$ns node]  
set n5 [$ns node]  
set n6 [$ns node]
```

```
$ns duplex-link $n1 $n5 0.011Mb 10ms DropTail  
$ns duplex-link $n2 $n3 1Mb 5ms DropTail  
$ns duplex-link $n2 $n4 1Mb 5ms DropTail
```



```
$ns duplex-link $n1 $n6 1Mb 5ms DropTail
$ns duplex-link $n3 $n6 1Mb 5ms DropTail
```

```
$ns duplex-link-op $n1 $n5 color red
$ns duplex-link-op $n2 $n3 color blue
```

```
$ns duplex-link-op $n1 $n5 orient left
$ns duplex-link-op $n2 $n3 orient right
```

```
$ns queue-limit $n1 $n5 50
$ns queue-limit $n2 $n3 0.011
$ns queue-limit $n2 $n4 50
$ns queue-limit $n1 $n6 50
$ns queue-limit $n3 $n6 50
```

```
$n1 color blue
$n2 color red
```

```
$n1 shape circle
$n2 shape box
```

```
set tcp1 [new Agent/TCP]
$ns attach-agent $n3 $tcp1
$tcp1 set fid_ 1
set tcpsink0 [new Agent/TCPSink]
$ns attach-agent $n5 $tcpsink0
$ns connect $tcp1 $tcpsink0
```

```
set tcp2 [new Agent/TCP]
$ns attach-agent $n4 $tcp2
set tcpsink1 [new Agent/TCPSink]
$ns attach-agent $n6 $tcpsink1
$ns connect $tcp2 $tcpsink1
```

```
set ftp0 [new Application/FTP]
set telnet0 [new Application/Telnet]
```

```
$ftp0 set packet-size 200
$ftp0 set Interval 0.005
$ftp0 attach-agent $tcp1
$ns at 0.5 "$ftp0 start"
$ns at 4.5 "$ftp0 stop"
```

```
$telnet0 set packet-size 200
$telnet0 set interval 0.005
$telnet0 attach-agent $tcp2
$ns at 0.5 "$telnet0 start"
$ns at 4.5 "$telnet0 stop"
```

```
$ns at 5.5 "finish"  
$ns run
```

3. Simulate an Ethernet LAN with 10 nodes consisting of TCP traffic, Telnet traffic generator.

```
set ns [new Simulator]  
set nf [open out.nam w]  
$ns namtrace-all $nf  
set tf [open out.tr w]  
$ns trace-all $tf
```

```
$ns color 1 red  
$ns color 2 brown
```

```
proc finish {} {  
    global ns nf tf  
    $ns flush-trace  
    close $nf  
    close $tf  
    exec nam out.nam &  
    exit(0)  
}
```

```
set n0 [$ns node]  
set n1 [$ns node]  
set n2 [$ns node]  
set n3 [$ns node]  
set n4 [$ns node]  
set n5 [$ns node]  
set n6 [$ns node]  
set n7 [$ns node]  
set n8 [$ns node]  
set n9 [$ns node]
```

```
$ns make-lan "$n0 $n1 $n2 $n3 $n4 $n5 $n6 $n7 $n8 $n9" 0.1Mb 0.1ms LL  
Queue/DropTail Mac/802_3
```

```
set tcp0 [new Agent/TCP]  
$ns attach-agent $n3 $tcp0  
$tcp0 set fid_ 1
```

```

set tcpsink0 [new Agent/TCPSink]
$ns attach-agent $n5 $tcpsink0
$ns connect $tcp0 $tcpsink0

set telnet0 [new Application/Telnet]
$telnet0 set packet-size 500
$telnet0 set interval 0.005
$telnet0 attach-agent $tcp0
$ns at 0.5 "$telnet0 start"
$ns at 2.0 "$telnet0 stop"

set tcp1 [new Agent/TCP]
$ns attach-agent $n3 $tcp1
$tcp1 set fid_ 2
set tcpsink0 [new Agent/TCPSink]
$ns attach-agent $n5 $tcpsink0
$ns connect $tcp1 $tcpsink0

set telnet1 [new Application/Telnet]
$telnet1 set packet-size 500
$telnet1 set interval 0.005
$telnet1 attach-agent $tcp1
$ns at 0.5 "$telnet1 start"
$ns at 2.0 "$telnet1 stop"

$ns at 5.0 "finish"
$ns run

```

4. Simulate an Ethernet LAN with 12 nodes consisting of multiple traffic(TCP and UDP).

- **Apply different path for multiple traffic**
- **Apply CBR,FTP,TELNET application**
- **Find the number of packets dropped.**
- **change the shape and color of every source and destination node.**
- **Change the color of packet for multiple Traffic**

```

set ns [new Simulator]
$ns color 1 brown
$ns color 2 green
set nf [open out.nam w]
$ns namtrace-all $nf
set tf [open out.tr w]
$ns trace-all $tf

```

```
proc finish {} {  
  global ns nf tf  
  $ns flush-trace  
  close $nf  
  close $tf  
  exec nam out.nam &  
  exit (0)  
}
```

```
set n0 [$ns node]  
set n1 [$ns node]  
set n2 [$ns node]  
set n3 [$ns node]  
set n4 [$ns node]  
set n5 [$ns node]  
set n6 [$ns node]  
set n7 [$ns node]  
set n8 [$ns node]  
set n9 [$ns node]  
set n10 [$ns node]  
set n11 [$ns node]
```

```
$ns make-lan "$n0 $n1 $n2 $n3 $n4 $n5 $n6 $n7 $n8 $n9 $n10 $n11" 0.22Mb 10ms LL  
Queue/DropTail Mac/802_3
```

```
$n0 color red  
$n1 color red  
$n2 color red  
$n3 color red  
$n4 color blue  
$n5 color blue  
$n6 color blue  
$n7 color blue  
$n8 color blue  
$n9 color purple  
$n10 color purple  
$n11 color purple
```

```
set udp1 [new Agent/UDP]  
$ns attach-agent $n0 $udp1  
set null1 [new Agent/Null]  
$ns attach-agent $n3 $null1  
$ns connect $udp1 $null1
```

```
set tcp1 [new Agent/TCP]  
$ns attach-agent $n4 $tcp1  
$tcp1 set fid_1
```

```

set tcpsink0 [new Agent/TCPSink]
$ns attach-agent $n8 $tcpsink0
$ns connect $tcp1 $tcpsink0

set tcp2 [new Agent/TCP]
$ns attach-agent $n9 $tcp2
$tcp2 set fid_ 2
set tcpsink1 [new Agent/TCPSink]
$ns attach-agent $n11 $tcpsink1
$ns connect $tcp2 $tcpsink1

set cbr1 [new Application/Traffic/CBR]
$cbr1 set packet-size 200
$cbr1 set interval 0.005
$cbr1 attach-agent $udp1
$ns at 0.5 "$cbr1 start"
$ns at 4.0 "$cbr1 stop"

set ftp0 [new Application/FTP]
$ftp0 set packet-size 200
$ftp0 set interval 0.005
$ftp0 attach-agent $tcp1
$ns at 0.5 "$ftp0 start"
$ns at 4.0 "$ftp0 stop"

set telnet0 [new Application/Telnet]
$telnet0 set packet-size 200
$telnet0 set interval 0.005
$telnet0 attach-agent $tcp2
$ns at 0.5 "$telnet0 start"
$ns at 4.5 "$telnet0 stop"

$ns at 5.0 "finish"
$ns run

```

5. Simulate the transmission of ping messages over a network topology consisting of 6 nodes.

```

set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set tf [open out.tr w]
$ns trace-all $tf

```

```

proc finish {} {
global ns nf tf
$ns flush-trace
close $nf
close $tf
exec nam out.nam &
exit(0)
}
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]

$ns duplex-link $n1 $n5 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 5ms DropTail
$ns duplex-link $n2 $n4 1Mb 5ms DropTail
$ns duplex-link $n1 $n6 1Mb 5ms DropTail
$ns duplex-link $n3 $n6 1Mb 5ms DropTail

$ns queue-limit $n1 $n5 50
$ns queue-limit $n2 $n3 50
$ns queue-limit $n2 $n4 50
$ns queue-limit $n1 $n6 50
$ns queue-limit $n3 $n6 50

Agent/Ping instproc recv {from rtt} {
$self instvar node_
puts "node [$node_ id] has recieved ping answer $from with round trip time $rtt
ms"}

set p1 [new Agent/Ping]
$ns attach-agent $n1 $p1

set p2 [new Agent/Ping]
$ns attach-agent $n3 $p2

$ns connect $p1 $p2
$ns at 0.2 "$p1 send"
$ns at 0.6 "$p2 send"
$ns at 0.4 "$p1 send"
$ns at 0.8 "$p2 send"

$ns at 1.0 "finish"
$ns run

```

6. Write a Program to implement a Ring topology using less number of statements in TCL Language apply PING Agent

```
set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set tf [open out.tr w]
$ns trace-all $tf

proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit(0)
}

set numnodes 5

for { set i 0 } {$i<$numnodes} {incr i} {
    set nodes($i) [$ns node]
}

for { set i 0 } {$i<$numnodes} {incr i} {
    set j [expr {($i+1)%$numnodes}]
    $ns duplex-link $nodes($i) $nodes($j) 1Mb 10ms DropTail }

Agent/Ping instproc recv {from rtt} {
    $self instvar node_
    puts "node [$node_ id] has received ping answer $from with round trip time of $rtt ms"}

set p1 [new Agent/Ping]
$ns attach-agent $nodes(0) $p1

set p2 [new Agent/Ping]
$ns attach-agent $nodes(2) $p2

$ns connect $p1 $p2

$ns at 0.2 "$p1 send"
$ns at 0.6 "$p2 send"
$ns at 0.4 "$p1 send"
$ns at 0.8 "$p2 send"

$ns at 1.0 "finish"
$ns run
```

7. Write a Program to implement a Star topology using less number of statements in TCL Language using UDP and TCP Agent

```
set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set tf [open out.tr w]
$ns trace-all $tf

$ns color 1 red
$ns color 2 blue

proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit(0)
}

set numnodes 7

for { set i 0 } {$i<$numnodes} {incr i} {
    set n($i) [$ns node]
}

for { set i 1 } {$i<7} {incr i} {
    $ns duplex-link $n(0) $n($i) 1Mb 10ms DropTail }

$ns duplex-link-op $n(0) $n(1) orient left-up
$ns duplex-link-op $n(0) $n(2) orient right-up

$ns duplex-link-op $n(0) $n(3) orient left
$ns duplex-link-op $n(0) $n(4) orient left-down

$ns duplex-link-op $n(0) $n(5) orient right
$ns duplex-link-op $n(0) $n(6) orient right-down

set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
$udp0 set fid_ 1
set null1 [new Agent/Null]
$ns attach-agent $n(2) $null1
$ns connect $udp0 $null1

set cbr0 [new Application/Traffic/CBR]
$cbr0 set packet-size 500
$cbr0 set interval 0.005
```



```
$cbr0 attach-agent $udp0  
$ns at 0.3 "$cbr0 start"  
$ns at 1.5 "$cbr0 stop"
```

```
set tcp0 [new Agent/TCP]  
$ns attach-agent $n(3) $tcp0  
$tcp0 set fid_ 2  
set tcpsink0 [new Agent/TCPSink]  
$ns attach-agent $n(5) $tcpsink0  
$ns connect $tcp0 $tcpsink0
```

```
set ftp0 [new Application/FTP]  
$ftp0 set packet-size 500  
$ftp0 set interval 0.01  
$ftp0 attach-agent $tcp0  
$ns at 0.5 "$ftp0 start"  
$ns at 2.0 "$ftp0 stop"
```

```
$ns at 2.5 "finish"  
$ns run
```